



VLSI Lab

CSE 405

Experiment Number: 04

Design and Implementation of a Modular 4-Bit Arithmetic Logic Unit (ALU) using Verilog in Xilinx ISE

Date: 04/10/2026

Submitted by:

Name : Md. Seeman Islam Seemanto
ID : 22201095
Section : C1
Semester : 4.2

Submitted to:

Md. Rasheduzzaman
Lecturer
Department of Computer Science and Engineering
University of Asia Pacific

Title of the Experiment

Hierarchical Design and Behavioral Simulation of a Modular 4-Bit Arithmetic Logical Unit (ALU) in Verilog HDL (Xilinx ISE 14.7 Platform).

Objectives

- To design a modular 4-bit Arithmetic Logic Unit (ALU) using a hierarchical, bottom-up design methodology in Verilog HDL.
- To implement fundamental arithmetic building blocks including a 1-bit Full Adder, a 4-bit Ripple Carry Adder, and a 2's complement 4-bit Subtractor.
- To construct bitwise logical sub-modules (AND, OR, XOR, NOT) and a 4-bit wide 8-to-1 Multiplexer for operation routing.
- To instantiate all functional sub-modules into a unified top-level ALU module (alu_top).
- To simulate and verify circuit functionality across diverse arithmetic and bitwise operation vectors using the ISim behavioral simulation environment.
- To analyze and evaluate synthesized RTL schematics and hierarchical structures generated by Xilinx Synthesis Technology (XST).
- To simulate and verify the correctness of the operations using ISim testbenches.

Theoretical Background

Combinational Logic & Arithmetic Logic Units (ALUs)

An Arithmetic Logic Unit (ALU) is a fundamental multi-operation combinational circuit that forms the computational core of a Central Processing Unit (CPU). It executes arithmetic calculations (such as addition and subtraction) and bitwise Boolean logic operations based on a set of control selection lines (ALU_Sel).

Arithmetic Operations & 2's Complement Subtraction

Binary addition is performed by chaining single-bit full adders where the carry-out (C_{out}) of each stage is propagated as the carry-in (C_{in}) to the next stage (Ripple Carry Adder). Binary subtraction ($A - B$) is implemented using 2's complement representation:

$$A - B = A + \bar{B} + 1$$

By bitwise-inverting input B ($\bar{B}$) and applying an initial carry-in $C_{in} = 1$ to the 4-bit adder module, subtraction is executed without requiring dedicated subtraction logic. The resulting borrows out (B_{out}) is given by the complement of the adder's carry-out ($\overline{C_{out}}$).

Operation Routing via Multiplexer

<i>ALU_Sel[2:0]</i>	<i>Selected Operation</i>	<i>Mathematical Expression</i>
3'b000	ADD	$A + B$
3'b001	SUB	$A - B$
3'b010	AND	$A \cdot B$
3'b011	OR	$A + B$
3'b100	XOR	$A \oplus B$
3'b101	NOT	$\bar{A}$
3'b110	Reserved	4'b0000
3'b111	Reserved	4'b0000

Hierarchical (Bottom-Up) Design Methodology

This experiment adopts a bottom-up hierarchical architecture:

1. **Leaf Modules:** full_adder, and_4bit, or_4bit, xor_4bit, not_4bit, and mux8to1_4bit.
2. **Intermediate Sub-blocks:** adder_4bit (instantiates four full_adder modules) and subtractor_4bit (instantiates adder_4bit).
3. **Top-Level Module:** alu_top instantiates all sub-blocks and routes inputs/outputs through the 8-to-1 multiplexer.

This modular structure simplifies debugging, enhances readability, and guarantees structural scalability.

Hardware and Software Requirements

- **Development Software:** Xilinx ISE Design Suite 14.7
- **Simulation Environment:** ISim Simulator
- **Synthesis Engine:** Xilinx Synthesis Technology
- **Virtualization Platform:** Oracle VirtualBox
- **Virtual Environment:** Oracle Linux 64-bit (
- **Host Operating System:** Windows 11 / Windows 10
- **Hardware Description Language:** Verilog HDL

- **Version Control & Portfolio Platform:** Git & GitHub Repository

Design Hierarchy and RTL Schematic

The structural design consists of three levels of modules:

Module Name	Description	Inputs	Outputs
full_adder	1-bit Full Adder primitive	a, b, cin	sum, cout
adder_4bit	4-bit Ripple Carry Adder (Instantiates 4x full_adder)	A[3:0], B[3:0], Cin	Sum[3:0], Cout
subtractor_4bit	4-bit Subtractor (Instantiates adder_4bit)	A[3:0], B[3:0]	Diff[3:0], Bout
and_4bit	4-bit Bitwise AND Unit	A[3:0], B[3:0]	Out[3:0]
or_4bit	4-bit Bitwise OR Unit	A[3:0], B[3:0]	Out[3:0]
xor_4bit	4-bit Bitwise XOR Unit	A[3:0], B[3:0]	Out[3:0]
not_4bit	4-bit Bitwise NOT Unit	A[3:0]	Out[3:0]
mux8to1_4bit	4-bit 8-to-1 Multiplexer	in0-in7 [3:0], sel[2:0]	out[3:0]
alu_top	Top-Level Modular ALU (Instantiates all units)	A[3:0], B[3:0], ALU_Sel[2:0]	ALU_Out[3:0], CarryOut

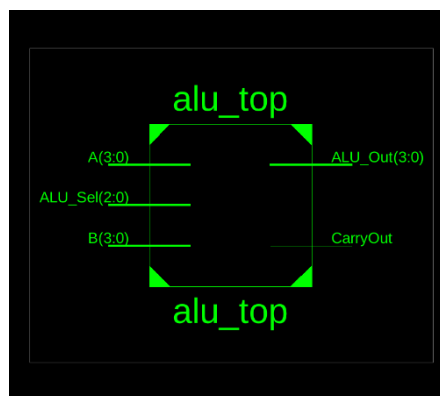


Figure 1: Top-level RTL schematic of the 4-bit Modular ALU

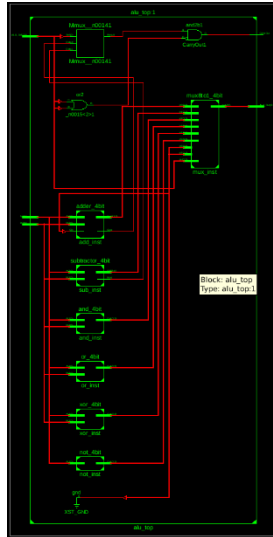


Figure 2: Expanded RTL schematic showing nested arithmetic and logic instances

Simulation and Result

The top-level ALU was verified using behavioral simulation in ISim. The testbench (tb_alu_top.v) instantiated alu_top as the unit under test (UUT), driving inputs *A*, *B*, and *ALU_Sel* across arithmetic and bitwise test vectors.

Time (ns)	Input A	Input B	ALU_Sel	Selected Operation	Expected Output	CarryOut
0–20	0000	0000	000	Default ADD	0000	0
20–40	1000 (8)	0101 (5)	000	ADD (8 + 5)	1101 (13)	0
40–60	1100 (12)	0100 (4)	001	SUB (12 – 4)	1000 (8)	0
60–80	1010	1100	010	Bitwise AND	1000	0
80–100	1010	0101	011	Bitwise OR	1111	0
100–120	1010	1100	100	Bitwise XOR	0110	0
120–140	1010	0000	101	Bitwise NOT (A)	0101	0

Comparing the simulated waveform against the expected arithmetic and logical behavior, all output vectors updated accurately according to the select line *ALU_Sel*, confirming functional correctness.

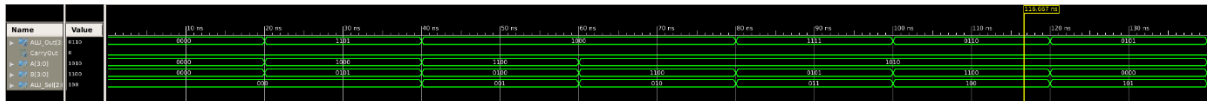


Figure 3: Behavioral simulation waveform of the 4-bit Modular ALU

Verilog Source Code

full_adder.v:

```
module full_adder (
    input a,
    input b,
    input cin,
    output sum,
    output cout
);
    assign sum = a ^ b ^ cin;
    assign cout = (a & b) | (b & cin) | (a & cin);
endmodule
```

adder_4bit.v:

```
module adder_4bit (
    input [3:0] A,
    input [3:0] B,
    input Cin,
    output [3:0] Sum,
    output Cout
);
    wire c1, c2, c3;

    full_adder fa0 (.a(A[0]), .b(B[0]), .cin(Cin), .sum(Sum[0]),
    .cout(c1));
    full_adder fa1 (.a(A[1]), .b(B[1]), .cin(c1), .sum(Sum[1]),
    .cout(c2));
    full_adder fa2 (.a(A[2]), .b(B[2]), .cin(c2), .sum(Sum[2]),
    .cout(c3));
    full_adder fa3 (.a(A[3]), .b(B[3]), .cin(c3), .sum(Sum[3]),
    .cout(Cout));
endmodule
```

subtractor_4bit.v:

```
module subtractor_4bit (  
    input [3:0] A,  
    input [3:0] B,  
    output [3:0] Diff,  
    output Bout  
);  
    wire [3:0] B_inv;  
    wire carry_out;  
  
    assign B_inv = ~B;  
  
    adder_4bit add_sub_inst (  
        .A(A),  
        .B(B_inv),  
        .Cin(1'b1),  
        .Sum(Diff),  
        .Cout(carry_out)  
    );  
  
    assign Bout = ~carry_out;  
endmodule
```

and_4bit.v:

```
module and_4bit (  
    input [3:0] A,  
    input [3:0] B,  
    output [3:0] Out  
);  
    assign Out = A & B;  
endmodule
```

or_4bit.v:

```
module or_4bit (  
    input [3:0] A,  
    input [3:0] B,  
    output [3:0] Out  
);  
    assign Out = A | B;  
endmodule
```

xor_4bit.v:

```
module xor_4bit (
    input [3:0] A,
    input [3:0] B,
    output [3:0] Out
);
    assign Out = A ^ B;
endmodule
```

not_4bit.v:

```
module not_4bit (
    input [3:0] A,
    output [3:0] Out
);
    assign Out = ~A;
endmodule
```

mux8to1_4bit.v:

```
module mux8to1_4bit (
    input [3:0] in0, in1, in2, in3, in4, in5, in6, in7,
    input [2:0] sel,
    output reg [3:0] out
);
    always @(*) begin
        case(sel)
            3'b000: out = in0;
            3'b001: out = in1;
            3'b010: out = in2;
            3'b011: out = in3;
            3'b100: out = in4;
            3'b101: out = in5;
            3'b110: out = in6;
            3'b111: out = in7;
            default: out = 4'b0000;
        endcase
    end
endmodule
```

alu_top.v:

```
module alu_top (
    input [3:0] A,
    input [3:0] B,
    input [2:0] ALU_Sel,
    output [3:0] ALU_Out,
    output CarryOut
);

    wire [3:0] add_res, sub_res, and_res, or_res, xor_res, not_res;
    wire add_cout, sub_bout;

    adder_4bit add_inst (.A(A), .B(B), .Cin(1'b0), .Sum(add_res),
    .Cout(add_cout));
    subtractor_4bit sub_inst (.A(A), .B(B), .Diff(sub_res),
    .Bout(sub_bout));

    and_4bit and_inst (.A(A), .B(B), .Out(and_res));
    or_4bit or_inst (.A(A), .B(B), .Out(or_res));
    xor_4bit xor_inst (.A(A), .B(B), .Out(xor_res));
    not_4bit not_inst (.A(A), .Out(not_res));

    mux8to1_4bit mux_inst (
        .in0(add_res), .in1(sub_res), .in2(and_res), .in3(or_res),
        .in4(xor_res), .in5(not_res), .in6(4'b0000), .in7(4'b0000),
        .sel(ALU_Sel), .out(ALU_Out)
    );

    assign CarryOut = (ALU_Sel == 3'b000) ? add_cout :
        (ALU_Sel == 3'b001) ? sub_bout : 1'b0;

endmodule
```

GitHub Repository Link

[The completed source code](#) (add)

Conclusion

In this experiment, a modular 4-bit Arithmetic Logic Unit (ALU) was successfully modeled using a bottom-up hierarchical architecture in Verilog HDL and verified within the Xilinx ISE 14.7 design suite. By instantiating basic full adders, logic gates, and a multiplexer under a top-level module, modular design scalability was effectively demonstrated. Functional simulation in ISim validated correct data output across all arithmetic and bitwise test vectors. Synthesis via XST generated an accurate RTL structure representing the nested sub-module hierarchy. This practical implementation solidifies fundamental concepts of digital system design and hardware description language modeling.